

Prompt:

Actúa como asistente de programación para una práctica de Estructura de Datos en Python.

Contexto:

Ya tengo un proyecto desarrollado en clase con Python, programación orientada a objetos y una

interfaz gráfica creada con Qt Designer. En el proyecto ya existe una clase Stack con los métodos

básicos de una pila. También existe una ventana o formulario donde se capturan datos y se muestran

resultados, así como la clase que realiza la carga y la detección del evento para convertir la

expression.

Objetivo:

Necesito agregar al proyecto existente la funcionalidad para convertir una expresión infija a

expresión posfija utilizando la estructura de pila ya creada en clase. Necesito se genere una clase

que contenga una pila y un método para llevar acabo la conversion.

Instrucciones importantes:

1. No generes un proyecto nuevo desde cero.
2. No reemplaces la clase Stack existente. Debes utilizarla e integrarte a su forma actual de trabajo.
3. Respeta la organización actual de archivos, clases, nombres de métodos y estilo de programación del proyecto.
4. Si necesitas proponer cambios, indícalos como cambios mínimos y explica exactamente en qué archivo y en qué parte deben colocarse.

5. La solución debe estar hecha con programación orientada a objetos.
6. La interfaz gráfica debe seguir usando el diseño generado con Qt Designer; no cambies el archivo .ui salvo que sea estrictamente necesario.
7. La aplicación debe permitir capturar una expresión infija, convertirla a posfija y mostrar el resultado en la interfaz.
8. Se asume que la expression infija capturada es válida ya que de otra forma la conversion puede fallar.
9. Usa operadores básicos: +, -, *, /, \$(Potencia) y paréntesis.
10. Explica brevemente cómo funciona el algoritmo y cómo se utiliza la pila en la conversion, para esto genera un archivo .md que contenga esta explicación.

Código existente:

```
[class Nodo:
```

```
    def __init__(self, dato):
        self.dato = dato
        self.next = None
```

```
class Stack:
```

```
    def __init__(self):
        self.head = None
```

```
    def push(self, dato):
        """Inserta un elemento en la parte superior (Top)"""
        nuevo_nodo = Nodo(dato)
        nuevo_nodo.next = self.head
```

```
self.head = nuevo_nodo
```

```
def pop(self):
```

```
    """Elimina y retorna el elemento en la parte superior (Top)"""
```

```
    if not self.head:
```

```
        return None
```

```
    dato = self.head.dato
```

```
    self.head = self.head.next
```

```
    return dato
```

```
def top(self):
```

```
    return self.head.dato if self.head else None
```

```
def print_stack(self):
```

```
    if not self.head:
```

```
        print("Stack: Empty")
```

```
    return
```

```
    nodos = []
```

```
    actual = self.head
```

```
    while actual:
```

```
        nodos.append(str(actual.dato))
```

```
        actual = actual.next
```

```
    print("Top -> " + " -> ".join(nodos) + " -> Base")
```

```
def is_empty(self):
```

```
        return self.head is None]

[from loads.load_menu import MenuListaEnlazada
import sys
from PyQt5.QtWidgets import QApplication
def main():
```

```
    app = QApplication(sys.argv)
    ventana = MenuListaEnlazada()
    ventana.show()
    sys.exit(app.exec_())
```

```
if __name__ == "__main__":
    main()
]
```

```
[<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
<class>Dialog</class>
<widget class="QDialog" name="Dialog">
<property name="geometry">
<rect>
<x>0</x>
<y>0</y>
<width>651</width>
<height>341</height>
</rect>
</property>
<property name="windowTitle">
```

```

    <string>Dialog</string>

</property>

<property name="styleSheet">
    <string notr="true">/* --- Paleta: Slate Blue & Cool Grey --- */
/* Fondo: #F8FAFC (Blanco azulado muy suave) */
/* Acento: #475569 (Azul Pizarra - sobrio y tecnológico) */

QDialog { background-color: #F8FAFC; }

QLabel { color: #334155; font-weight: 600; }

QLineEdit {
    background-color: #FFFFFF;
    border: 1px solid #CBD5E1;
    color: #1E293B;
}

QLineEdit:focus { border: 1.5px solid #475569; }

QPushButton {
    background-color: #475569;
    color: #F1F5F9;
}

QPushButton:hover { background-color: #334155; }</string>

</property>

<widget class="QLabel" name="label">
    <property name="geometry">
        <rect>

```

```
<x>70</x>
<y>70</y>
<width>101</width>
<height>51</height>
</rect>
</property>
<property name="text">
  <string>Expresión Infija:</string>
</property>
<property name="alignment">
  <set>Qt::AlignCenter</set>
</property>
</widget>
<widget class="QLineEdit" name="lineEdit">
  <property name="geometry">
    <rect>
      <x>180</x>
      <y>70</y>
      <width>401</width>
      <height>41</height>
    </rect>
  </property>
</widget>
<widget class="QPushButton" name="pushButton">
  <property name="geometry">
    <rect>
      <x>140</x>
```

```
<y>160</y>
<width>121</width>
<height>41</height>
</rect>
</property>
<property name="text">
  <string>Realizar conversión</string>
</property>
</widget>
<widget class="QLabel" name="label_2">
  <property name="geometry">
    <rect>
      <x>150</x>
      <y>260</y>
      <width>381</width>
      <height>41</height>
    </rect>
  </property>
  <property name="text">
    <string>Resultado...</string>
  </property>
  <property name="alignment">
    <set>Qt::AlignCenter</set>
  </property>
</widget>
</widget>
<resources/>
```

```
<connections/>
```

```
</ui>
```

```
]
```

Resultado esperado de tu respuesta:

- Primero analiza la estructura del código existente.
- Después propón la clase con el método para convertir de infija a posfija.
- Luego indica cómo conectar ese método con el botón o evento de la interfaz gráfica.
- Finalmente proporciona pruebas para comprobar que funciona.
- No omitas explicación, porque debo entender el código para defenderlo en clase ante el profesor y mis compañeros.

Resultado de la IA:

Para integrar la funcionalidad de conversión de infija a posfija en tu proyecto actual, seguiremos un enfoque limpio que aprovecha tu clase Stack existente.

1. Análisis de la Estructura Existente

Tu código utiliza una implementación de pila basada en nodos. El algoritmo Shunting-Yard (de Dijkstra), que usaremos para la conversión, se adapta perfectamente a esta estructura, ya que solo requiere las operaciones push, pop, top e is_empty que ya tienes implementadas.

2. Clase de Conversión

Crearemos un nuevo archivo llamado fijainfija.py. Esta clase encapsulará la lógica de precedencia y el uso de la Stack.

Python

```
from estructuras.lineales. stack import Stack  
  
# Asegúrate de que tu archivo se llame stack.py
```



```

class InfijaAPosfija:

    def __init__(self):

        self.precedencia = {'+': 1, '-': 1, '*': 2, '/': 2, '$': 3}

    def convertir(self, expresion):

        pila = Stack()

        posfija = ""

        for char in expresion:

            # Si es operando, añadir a la salida
            if char.isalnum():

                posfija += char

            # Si es paréntesis de apertura, pushear a la pila
            elif char == '(':

                pila.push(char)

            # Si es paréntesis de cierre, vaciar hasta encontrar apertura
            elif char == ')':

                while not pila.is_empty() and pila.top() != '(':

                    posfija += pila.pop()

                pila.pop() # Eliminar el '('

            # Si es operador
            else:

                while not pila.is_empty() and pila.top() != '(' and \

                    self.precedencia.get(pila.top(), 0) >= self.precedencia.get(char, 0):

                    posfija += pila.pop()

                pila.push(char)

```

```
# Vaciar lo que quede en la pila
while not pila.is_empty():
    posfija += pila.pop()

return posfija
```

3. Integración con la Interfaz (Qt Designer)

En el archivo donde tienes la lógica de tu ventana (probablemente load_menu.py o similar), debes conectar el botón a una función que utilice el InfijaAPosfija.

Cambios mínimos:

1. Importa la clase InfijaAPosfija.
2. Dentro de la clase de tu ventana, conecta el botón:

Python

En la parte superior del archivo de la ventana

```
from conversor import InfijaAPosfija
```

En el método __init__ o setupUi de tu clase de ventana:

```
self.pushButton.clicked.connect(self.procesar_conversion)
```

Agrega este método a tu clase:

```
def procesar_conversion(self):
    expresion = self.lineEdit.text()
    conversor = InfijaAPosfija()
    resultado = conversor.convertir(expresion)
    self.label_2.setText(f"Resultado: {resultado}")
```

4. Explicación del Algoritmo (explicacion.md)

Markdown

Algoritmo de Conversión: Infija a Posfija

El algoritmo utiliza una **Pila (Stack)** para gestionar los operadores y paréntesis, manteniendo el orden de precedencia correcto.

1. **Operandos:** Se envían directamente a la cadena de resultado.
2. **Paréntesis '(':** Se apilan inmediatamente para marcar el inicio de un sub-bloque.
3. **Paréntesis ')':** Se desapilan todos los operadores de la pila hasta encontrar el '(' correspondiente, enviándolos al resultado.
4. **Operadores (+, -, *, /, \$):** - Mientras el operador actual tenga menor o igual precedencia que el operador que está en el **top** de la pila, se desapila el contenido de la pila y se envía al resultado.
 - Finalmente, se apila el operador actual.
5. **Finalización:** Al terminar la expresión, se desapilan todos los operadores restantes en la pila hacia el resultado.

Le cuestione sobre el archivo load para cargar la interfaz

Para completar la integración, necesitas el archivo que gestiona la lógica de la ventana y los eventos. Basado en tu estructura original, este archivo debe llamarse `load_menu.py` (o similar, según tu proyecto).

Aquí tienes el código necesario para conectar la interfaz con la lógica de conversión que definimos anteriormente.

Archivo: `loads/load_menu.py`

Debes asegurarte de que tu clase herede de la interfaz que cargaste con Qt Designer.

Python

```
from PyQt5.QtWidgets import QDialog
```

```
from PyQt5 import uic
```

```
from conversor import InfijaAPosfija # Importamos la clase que creamos antes
```

```

class MenuListaEnlazada(QDialog):

    def __init__(self):
        super().__init__()

        # Carga el archivo .ui (asegúrate de que la ruta sea correcta)
        uic.loadUi("ruta/a/tu/archivo_diseno.ui", self)

        # Conexión del botón al evento de conversión
        self.pushButton.clicked.connect(self.procesar_conversion)

    def procesar_conversion(self):

        # 1. Obtener la expresión desde el QLineEdit
        expresion = self.lineEdit.text()

        # 2. Instanciar el conversor
        conversor = InfijaAPosfija()

        # 3. Realizar la conversión
        resultado = conversor.convertir(expresion)

        # 4. Mostrar el resultado en el QLabel (label_2)
        self.label_2.setText(f"Resultado: {resultado}")

```

Notas importantes para tu implementación:

1. Ruta del archivo .ui: En la línea `uic.loadUi(..., self)`, verifica que la ruta apunte correctamente a donde tienes guardado tu archivo de Qt Designer.
2. Organización: Si tu clase `Stack` y `Nodo` están en otro archivo (por ejemplo, `estructuras.py`), asegúrate de que el `import` en `conversor.py` sea el correcto:
 - Ejemplo: `from estructuras import Stack`

3. El archivo conversor.py: Asegúrate de que este nuevo archivo esté en la raíz de tu proyecto o en la carpeta donde load_menu.py pueda importarlo.

File "c:\Users\walmart
clouthier\Documents\programacion2\proyectos\estructuraDatos\main.py", line 12,
in <module>

```
main()
```

File "c:\Users\walmart
clouthier\Documents\programacion2\proyectos\estructuraDatos\main.py", line 7, in
main

```
ventana = MenuListaEnlazada()
```

File "c:\Users\walmart
clouthier\Documents\programacion2\proyectos\estructuraDatos\loads\load_menu.p
y", line 14, in __init__

```
self.pushButton.clicked.connect(self.procesar_conversion)
```

AttributeError: 'MenuListaEnlazada' object has no attribute 'pushButton'

El error AttributeError: 'MenuListaEnlazada' object has no attribute 'pushButton' ocurre porque, aunque cargaste el archivo .ui con uic.loadUi(), el objeto MenuListaEnlazada aún no tiene mapeados los nombres de los widgets (como pushButton) como atributos directos de la clase.

La Solución

Cuando usas uic.loadUi(), los nombres de los widgets definidos en el archivo XML (tu archivo .ui) se cargan dinámicamente. Sin embargo, para acceder a ellos como self.nombre_del_widget, debes asegurarte de que el archivo se cargó correctamente y que los nombres coincidan exactamente.

Revisa y ajusta tu loads/load_menu.py siguiendo estos pasos:

1. Verifica los nombres en Qt Designer

Abre tu archivo .ui en **Qt Designer** y asegúrate de que, en el panel de **Object Inspector** (arriba a la derecha), los nombres sean exactamente los mismos que escribiste:

- El botón debe llamarse pushButton.
- El campo de texto debe llamarse lineEdit.
- La etiqueta de resultado debe llamarse label_2.

2. Ajuste en loads/load_menu.py

Asegúrate de conectar la señal **después** de que el archivo .ui haya terminado de cargarse. Aquí tienes el código corregido:

Python

```
from PyQt5.QtWidgets import QDialog
```

```
from PyQt5 import uic
```

```
import os # Importamos os para manejar rutas relativas
```

```
# Asegúrate de importar tu clase desde donde esté guardada
```

```
from conversor import InfijaAPosfija
```

```
class MenuListaEnlazada(QDialog):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        # Construye la ruta de forma segura para evitar errores de ubicación
```

```
        base_path = os.path.dirname(__file__)
```

```
        ui_path = os.path.join(base_path, "..", "tu_archivo.ui") # Ajusta el nombre del  
archivo .ui
```

```
        # Cargar la interfaz
```

```
        uic.loadUi(ui_path, self)
```

```
        # AHORA que el archivo está cargado, los atributos ya existen
```

```
        # Verificamos que existan antes de conectar para evitar el error
```

```
        if hasattr(self, 'pushButton'):
```

```
            self.pushButton.clicked.connect(self.procesar_conversion)
```

```
        else:
```

```
print("Error: No se encontró el widget 'pushButton' en el archivo .ui")
```

```
def procesar_conversion(self):  
    expresion = self.lineEdit.text()  
    conversor = InfijaAPosfija()  
    resultado = conversor.convertir(expresion)  
    self.label_2.setText(f"Resultado: {resultado}")
```

¿Por qué sucede esto?

1. **Instanciación temprana:** Si intentas acceder a `self.pushButton` antes de la línea `uic.loadUi`, Python no sabe que ese objeto existe.
2. **Nombres distintos:** Si en Qt Designer el botón se llama de otra forma (por ejemplo, `btnConvertir`), el código fallará al buscar `pushButton`.
 - Tip: Si no estás seguro de los nombres, puedes imprimir `print(self.__dict__.keys())` justo después de `uic.loadUi(ui_path, self)` para ver qué nombres de variables ha registrado PyQt.

¿Qué hacer si sigue fallando? Si el error persiste, abre tu archivo `.ui` con un editor de texto (como el Bloc de notas) y busca la línea que dice `<widget class="QPushButton" name="pushButton">`. Si el `name` es diferente a `pushButton`, cámbialo ahí mismo o cámbialo en Qt Designer y guarda el archivo.

Traceback (most recent call last):

```
File "c:\Users\walmart  
clouthier\Documents\programacion2\proyectos\estructuraDatos\loads\load_menu.p  
y", line 25, in procesar_conversion
```

```
    expresion = self.lineEdit.text()
```

AttributeError: 'MenuListaEnlazada' object has no attribute 'lineEdit'

El error persiste porque, al igual que con el botón, el atributo `lineEdit` no está siendo reconocido como parte de tu clase `MenuListaEnlazada`. Esto sucede casi siempre por uno de estos dos motivos: el archivo `.ui` no se está cargando correctamente o los nombres definidos en Qt Designer no coinciden con los que estás usando en el código.

Para solucionar esto de manera definitiva y profesional, vamos a aplicar un **proceso de depuración** para que puedas ver exactamente qué está pasando.

1. Depuración del error (Haz esto primero)

Agrega estas líneas justo después de `uic.loadUi(...)` en tu archivo `loads/load_menu.py` para imprimir qué objetos reconoce realmente PyQt:

Python

```
uic.loadUi(ui_path, self)

# DEBUG: Imprime todos los atributos disponibles en la clase
print("Widgets encontrados en la interfaz:")
for attr in dir(self):
    if isinstance(getattr(self, attr), type): # Solo queremos ver widgets
        continue
    print(f"- {attr}")
```

Al ejecutar, revisa la consola:

- Si no ves `lineEdit` ni `pushButton` en la lista, **tu archivo .ui no se está cargando** (o es un archivo vacío/diferente).
- Si ves nombres similares como `lineEdit_2` o `txt_expresion`, es que en Qt Designer tienen nombres distintos. **Debes usar el nombre exacto que aparezca en esa lista.**

2. Corrección del archivo `loads/load_menu.py`

Suponiendo que el archivo sí carga pero los nombres no coinciden, asegúrate de que la estructura sea esta:

Python

```
from PyQt5.QtWidgets import QDialog
from PyQt5 import uic
import os
from conversor import InfijaAPosfija
```

```
class MenuListaEnlazada(QDialog):
```

```
    def __init__(self):
```



```

super().__init__()

# Ruta dinámica al archivo .ui
base_path = os.path.dirname(__file__)

# Asegúrate de que el nombre coincida con tu archivo real
ui_path = os.path.join(base_path, "..", "tu_archivo.ui")

uic.loadUi(ui_path, self)

# Conexión
self.pushButton.clicked.connect(self.procesar_conversion)

def procesar_conversion(self):
    # Verifica nuevamente el nombre en Qt Designer.
    # Si ahí dice 'lineEdit', debe funcionar. Si dice otra cosa, cámbialo aquí.
    try:
        expresion = self.lineEdit.text()
        conversor = InfijaAPosfija()
        resultado = conversor.convertir(expresion)
        self.label_2.setText(f"Resultado: {resultado}")
    except AttributeError:
        print("ERROR: El nombre del widget en tu archivo .ui no es 'lineEdit'")

```

3. Checklist de verificación en Qt Designer

Si sigue fallando, abre tu archivo .ui en Qt Designer y realiza este checklist rápido:

1. Haz clic en el cuadro de texto.
2. Mira el **Object Inspector** (arriba a la derecha).
3. ¿El nombre (objectName) es exactamente lineEdit?

- **Si dice otra cosa:** Cámbialo ahí mismo en la propiedad `objectName`.
- **Si no guardaste los cambios:** Asegúrate de presionar `Ctrl + S` en Qt Designer después de cambiar los nombres.

¿Qué nombre aparece en el "Object Inspector" de Qt Designer para el cuadro de texto y para el botón? Si me confirmas esos nombres, puedo darte el código exacto que funcionará a la primera.

mi funcion debe de quedar asi, la que va ligada al `load_menu` def `abrir_ventana_pilas(self)`:

```
self.ventana_pilas = DialogoPilas()

self.ventana_pilas.exec_()
```

Entendido. Si tu estructura actual utiliza una ventana principal (el menú) que abre un **diálogo** (la ventana de la pila) mediante `abrir_ventana_pilas`, entonces la lógica de la conversión debe residir **dentro** de la clase `DialogoPilas`.

Aquí tienes cómo estructurar tu archivo donde definiste la clase `DialogoPilas` para que funcione correctamente:

Ajuste en `dialogo_pilas.py` (o donde esté la clase)

Debes asegurarte de que `uic.loadUi` se ejecute en el `__init__` de esa clase específica. Así, cuando llames a `abrir_ventana_pilas`, la interfaz se cargará y los widgets estarán disponibles para la lógica.

Python

```
from PyQt5.QtWidgets import QDialog

from PyQt5 import uic

from conversor import InfijaAPosfija # Tu clase lógica
```

```
class DialogoPilas(QDialog):

    def __init__(self):
        super().__init__()

        # Carga el archivo .ui de tu formulario
        uic.loadUi("tu_archivo.ui", self)
```

```
# Ahora que el UI está cargado, los objetos (lineEdit, etc.) ya existen
self.pushButton.clicked.connect(self.procesar_conversion)
```

```
def procesar_conversion(self):
    # Capturamos la entrada
    expresion = self.lineEdit.text()

    # Instanciamos el conversor
    conversor = InfijaAPosfija()

    # Realizamos la conversión
    resultado = conversor.convertir(expresion)

    # Actualizamos la etiqueta de resultado
    self.label_2.setText(str(resultado))
```

¿Por qué esto soluciona el `AttributeError`?

1. **Contexto del `__init__`:** Al ejecutar `self.ventana_pilas = DialogoPilas()`, se llama al constructor de la clase. Es dentro de **ese** constructor donde `uic.loadUi(..., self)` mapea los elementos (`lineEdit`, `pushButton`) a la instancia `self`.
2. **Acceso directo:** Una vez cargada la interfaz, `self.lineEdit` y `self.pushButton` pasan a ser atributos válidos de `self` (el objeto `DialogoPilas`).
3. **Seguridad:** Como la conexión del botón se realiza dentro del `__init__` de la misma clase, se garantiza que los objetos existan antes de que el usuario interactúe con ellos.

Resumen de la estructura de archivos

- **main.py:** Lanza el menú principal.
- **load_menu.py:** Tiene el método `abrir_ventana_pilas` que instancia `DialogoPilas`.

- **dialogo_pilas.py**: Contiene la clase DialogoPilas (con el código que te puse arriba).
- **conversor.py**: Contiene la lógica matemática (la clase InfijaAPosfija).
- **stack.py**: Tu clase Pila original.

Con este cambio, al presionar el botón en el menú principal, se abrirá la ventana correctamente y, al escribir en el campo y presionar "Realizar conversión", la lógica de la pila procesará la expresión sin errores de atributos. ¿Logras ver el formulario al ejecutar el código ahora?

```
por que no me queda from PyQt5.QtWidgets import QMainWindow
```

```
from PyQt5 import uic
```

```
from loads.load_lista_enlazada_simple import DialogoListaEnlazada
```

```
from loads.load_dialogo_pila import DialogoPilas
```

```
from loads.load_conversion_fijainfija import InfijaAPosfija
```

```
class MenuListaEnlazada(QMainWindow):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        uic.loadUi("ui/ListaEnlazada.ui", self)
```

```
        self.Lista_Enlazada.triggered.connect(self.abrir_ventana_lista)
```

```
        self.actionPilas.triggered.connect(self.abrir_ventana_pilas)
```

```
        self.actionConversion.triggered.connect(self.procesar_conversion)
```

```
    def abrir_ventana_lista(self):
```

```
        self.ventana_lista = DialogoListaEnlazada()
```

```
        self.ventana_lista.exec_()
```

```
    def abrir_ventana_pilas(self):
```

```

        self.ventana_pilas = DialogoPilas()

        self.ventana_pilas.exec_()

def procesar_conversion(self):

    self.conversor = InfijaAPosfija()

    self.conversor.exec_()from PyQt5.QtWidgets import QMainWindow
from PyQt5 import uic
from loads.load_lista_enlazada_simple import DialogoListaEnlazada
from loads.load_dialogo_pila import DialogoPilas
from loads.load_conversion_fijainfija import InfijaAPosfija

class MenuListaEnlazada(QMainWindow):

    def __init__(self):
        super().__init__()
        uic.loadUi("ui/ListaEnlazada.ui", self)

        self.Lista_Enlazada.triggered.connect(self.abrir_ventana_lista)
        self.actionPilas.triggered.connect(self.abrir_ventana_pilas)
        self.actionConversion.triggered.connect(self.procesar_conversion)

def abrir_ventana_lista(self):

    self.ventana_lista = DialogoListaEnlazada()

    self.ventana_lista.exec_()

def abrir_ventana_pilas(self):

    self.ventana_pilas = DialogoPilas()

```

```
self.ventana_pilas.exec_()
```

```
def procesar_conversion(self):
```

```
    self.conversor = InfijaAPosfija()
```

```
    self.conversor.exec_()
```

El problema principal es una **confusión de arquitectura**. Estás tratando de usar una clase que contiene la lógica de conversión (InfijaAPosfija) como si fuera una ventana de interfaz gráfica (QDialog).

El error fundamental

En tu código, haces esto:

Python

```
self.conversor = InfijaAPosfija()
```

```
self.conversor.exec_() # <--- ERROR: InfijaAPosfija no es una ventana
```

El método `.exec_()` solo existe en clases que heredan de QDialog. Si tu clase InfijaAPosfija es la que tiene la lógica de conversión (la clase que escribimos al principio), **esa clase no debe ser una ventana**.

La solución correcta

Debes separar la **Ventana de Conversión** de la **Lógica de Conversión**.

1. **Crea una clase ventana nueva** (llamémosla VentanaConversion) que cargue el `.ui`.
2. **Usa la clase lógica** (InfijaAPosfija) dentro de esa ventana.

Paso 1: Crea loads/load_conversion_fijainfija.py

Esta clase **debe** heredar de QDialog para poder usar `exec_()`.

Python

```
from PyQt5.QtWidgets import QDialog
```

```
from PyQt5 import uic
```

```
from conversor import InfijaAPosfija # Tu clase lógica (sin UI)
```

```

class VentanaConversion(QDialog):

    def __init__(self):
        super().__init__()

        uic.loadUi("ui/TuArchivoDeConversion.ui", self) # Asegúrate de que este
        archivo exista

        self.pushButton.clicked.connect(self.ejecutar_logica)

    def ejecutar_logica(self):
        # 1. Obtener datos
        expresion = self.lineEdit.text()

        # 2. Usar la clase lógica que creamos antes
        conversor = InfijaAPosfija()
        resultado = conversor.convertir(expresion)

        # 3. Mostrar resultado
        self.label_2.setText(resultado)

```

Paso 2: Actualiza tu MenuListaEnlazada

Ahora, en tu menú principal, simplemente llamas a esta nueva clase ventana:

Python

En tus imports

```
from loads.load_conversion_fijainfija import VentanaConversion
```

... dentro de MenuListaEnlazada ...

```

def procesar_conversion(self):
    # Aquí instancia la ventana, NO la lógica directamente
    self.ventana_conversor = VentanaConversion()

```

```
self.ventana_conversor.exec_()
```

Resumen de por qué fallaba:

- **AttributeError**: Si tu clase InfijaAPosfija no hereda de QDialog, no tiene métodos como `exec_()`, ni conoce nada sobre `lineEdit` o `pushButton`.
- **Regla de oro**: * QDialog / QMainWindow son para **mostrar cosas** (botones, campos de texto).
 - Tu clase InfijaAPosfija es para **hacer cálculos** (lógica matemática).
 - **Nunca** intentes llamar `.exec_()` a una clase de lógica pura.

Para entender mejor cómo se separan estas responsabilidades en una aplicación profesional, este diagrama te ayudará:

¿Tienes el archivo `.ui` específico para la conversión, o planeabas usar el mismo formulario de las pilas?

cual es el significado de esto `isalnum()`

En Python, el método `.isalnum()` es una función integrada para cadenas de texto (strings) que sirve para validar qué tipo de caracteres contiene dicha cadena.

Su nombre es una abreviación de **"is alphanumeric"** (¿es alfanumérico?).

¿Qué significa exactamente?

El método devuelve un valor booleano:

- **True** (Verdadero): Si **todos** los caracteres en la cadena son letras (a-z, A-Z) o números (0-9).
- **False** (Falso): Si la cadena contiene al menos un carácter que no sea ni letra ni número (por ejemplo: espacios, símbolos como `+`, `-`, `*`, `/`, o signos de puntuación).

Ejemplos prácticos:

Python

Ejemplos que devuelven True

```
print("Hola123".isalnum()) # True: solo letras y números
```

```
print("Python".isalnum()) # True: solo letras
```

Ejemplos que devuelven False


```
print("Hola Mundo".isalnum()) # False: el espacio no es alfanumérico
```

```
print("A+B".isalnum())      # False: el símbolo '+' no es alfanumérico
```

```
print("123!".isalnum())     # False: el símbolo '!' no es alfanumérico
```

¿Por qué lo usamos en tu código de conversión?

En el algoritmo que estamos implementando para convertir de infija a posfija, necesitamos distinguir entre **operandos** (los datos) y **operadores** (la lógica):

1. Cuando el algoritmo recorre la expresión, `char.isalnum()` actúa como un **filtro**:
 - Si es True (una letra o número), significa que es un **operando** (como 'A', 'B', '5'), por lo tanto, lo enviamos directamente a nuestra cadena de resultado.
 - Si es False, significa que el carácter es un **operador** ('+', '*', etc.) o un paréntesis, por lo que debe pasar por el proceso de la pila para verificar su prioridad.

Es una forma muy eficiente y legible de clasificar los elementos de la expresión sin tener que escribir una lista larga de validaciones manuales.

¿Te ha quedado clara la distinción entre operandos y operadores basada en esta validación?